

HR AUTOMATION DASHBOARD

Standard Operating Procedure

Techolution — India & US Workforce

Built by: Lokineni Hindhuja

Date: 14 June 2026

Stack: Google Sheets + Apps Script + HtmlService

Primary Access: hr@techolution.com

Table of Contents

- 1. System Overview & Architecture
- 2. Source Data Tabs
- 3. Configuration (_Config Tab)
- 4. How HR Uses the Sheet Dashboard Daily
- 5. How HR Uses the Web App Daily
- 6. Adding Employees, Departments & Projects
- 7. Real-Time Reflection Pattern
- 8. Header-Agnostic Column Lookup
- 9. In-Memory Cache Layer
- 10. Input Validation
- 11. Alerts System
- 12. Org Chart
- 13. Drill-Down Filters
- 14. Email Digest
- 15. Automation & Triggers
- 16. Logs, Changelog & EmailLog
- 17. Modifying Thresholds & Config
- 18. Troubleshooting Guide
- 19. Change Management & Version Control
- 20. Apps Script Code Maintenance

1. System Overview & Architecture

The HR Automation Dashboard is a production-grade workforce intelligence system built for Techolution's India and US teams. It runs entirely on the Google Workspace stack, with no external servers and no third-party SaaS. HR operators interact only with familiar Google Sheets, while the system automatically computes, renders, and alerts in real time across two surfaces.

Two Surfaces

- **Sheet Dashboard** — a dedicated Dashboard tab inside the Google Sheets workbook. Every cell is written by Apps Script code, not by spreadsheet formulas. The dashboard is fully reproducible from source data plus code.
- **Web App** — a deployed Apps Script web app available at a Google URL. HR signs in with their Google account and views a browser-based executive dashboard that mirrors the Sheet dashboard, with auto-refreshing data (see Section 7.5).

Architecture Diagram

The diagram below shows how data flows from the source tabs, through the Apps Script processing layer, to the two output surfaces and the email digest.

Layer	Component	Responsibility
INPUT	India Employee Database	Employee records for India workforce
INPUT	US Employee Database	Employee records for US workforce (different column order)
INPUT	RM Data	Monthly resource allocation by employee
INPUT	Finance	CTC and financial data per employee
INPUT	Productivity	Average hours/day and below-target flags
INPUT	Risk Report	HR-maintained employee risk records
INPUT	Offboarded Resources	All employee exits, used for attrition
INPUT	_Config	All tunable thresholds and settings
SCRIPT	Data Layers	normalizeHeader, getSheetData, cachedEmployees, getField, joinOn, in-memory cache getConfig reads _Config; logInfo/logWarn/logError write to Logs tab
SCRIPT	Config.gs + Logs.gs	getLWDAAlerts, getProbationAlerts, sendAlertDigest, logEmailStatus
SCRIPT	Alerts.gs	renderDashboard — writes the Sheet Dashboard tab
SCRIPT	Dashboard.gs	renderOrgChart — rebuilds the hierarchical org chart tab
SCRIPT	OrgChart.gs	setupDrillDownFilters, renderDrillDown,
SCRIPT	DrillDown.gs	getUniqueValues, setDropdown onEditInstallable, refreshAll, validateEmployeeEdit,
SCRIPT	Triggers.gs	dailyPipelineRun, writeChangelog, createInstallableTriggers
SCRIPT	WebApp.gs	doGet, getWebAppData, getQuarterlyAttrition, buildOrgData, formatDate

Layer	Component	Responsibility
OUTPUT	Dashboard tab	Sheet-rendered KPIs, alerts, breakdowns, risk register, productivity flags
OUTPUT	OrgChart tab	Text-based indented hierarchy tree
OUTPUT	DrillDown tab	Filterable employee table with dropdown controls
OUTPUT	Logs / Changelog tabs	Execution history and daily run summaries
OUTPUT	Web App URL	Browser dashboard, auto-refreshing every 1.5 seconds
OUTPUT	Email digest	Daily HTML summary sent to hr@techolution.com

Note: All script-layer components read exclusively through DataLayer.gs and Config.gs — no file accesses a sheet directly using hardcoded ranges or column letters.

2. Source Data Tabs

All data lives in Google Sheets. HR edits these tabs directly, and the system reads from them on every trigger or page load. No data is duplicated outside the spreadsheet.

Tab	Contents
India Employee Database	Employee ID, Employee Name, Department, Designation, Reporting Manager, Skillset, Date of Joining, Employment Status, Last Working Day (interns only). Region is set to 'India' by the script.
US Employee Database	Same core fields as India, plus Current Allocation (%) at column 6 and CTC (Annual USD) at column 7. Column order differs from the India tab and is handled by the positional fallback map (see Section 8).
RM Data	Monthly resource allocation by employee. New month columns are added each month; the script auto-discovers the current month column without any hardcoded month name.
Finance	CTC (INR and USD, annual and monthly) and related financial data per employee.
Productivity	Overall Avg Hrs/Day and Below 8 Hrs Flag per employee. Drives the Productivity Flags KPI and dashboard section.
Risk Report	Maintained by HR. Employee Name, Risk Category, Risk Level, Status, Mitigation Action. Edits surface on both surfaces immediately via the onEdit pipeline.
Offboarded Resources	All employee exits. Used for live quarterly attrition calculation. The Last Working Day field determines which quarter each exit belongs to.
_Con f ig	All tunable thresholds and settings. See Section 3 for the full list.

3. Configuration (_Config Tab)

All thresholds live in the _Config tab and are never hardcoded in any script file. To change any value, an admin or HR user edits column B in _Config — no script changes are needed.

Key	Current Value	Purpose
LW D_ALERT_DAYS	45	Days before or after an intern's Last Working Day to surface LWD alerts (covers both upcoming and just-passed)

Key	Current Value	Purpose
		exists), Days before the confirmation date to surface probation alerts.
PROBATION_ALERT_DAYS	30	Length of the probation period in days.
PROBATION_PERIOD_DAYS	180	Confirmation date = Date of Joining + 180.
PRODUCTIVITY_TARGET_HRS	8	Target hours per day. Employees averaging below this are flagged.
ALERT_EMAIL_RECIPIENT	hr@techolution.com	Email address(es) for the daily alert digest. Separate multiple recipients with commas.
COMPANY_NAME	Techolution	Company name used in dashboard headers and email subject lines.

Note: Never hardcode these values inside .gs files. Always read them via getConfig(). The pipeline validates that required keys exist and falls back to safe defaults if a key is missing, logging a warning to the Logs tab.

4. How HR Uses the Sheet Dashboard Daily

The Sheet dashboard is fully automatic. HR never runs a script, never clicks Refresh, and never opens the Apps Script editor. The system does everything in response to normal data entry.

Dashboard Tab Sections

Section	What It Shows
HR Analytics Dashboard header	Title and generated timestamp. Updates automatically on every refresh.
Key Metrics	Total employees, India count, US count, interns, under probation, offboarded, high risk cases, productivity flags.
Alert Summary	Count of active LWD alerts and active probation alerts.
Department Breakdown	Headcount per department, sorted from highest to lowest.
Region Breakdown	India versus US headcount.
Recent Alerts	Alert type (LWD or Probation), employee name, and current alert status.
Risk Register	All active risk records — employee, category, level, status, and mitigation action.
Productivity Flags	Employees currently flagged as below the target hours per day.

OrgChart Tab

A hierarchical org chart rebuilt from the Reporting Manager column on every refresh. Manager changes are reflected instantly — there is no manual redrawing. Managers who are not listed as a direct report under anyone else appear as root nodes at the top of the tree.

DrillDown Tab

HR can filter the employee list by Region, Department, Reporting Manager, and Employment Status using the dropdowns in row 2. Results update automatically when a dropdown value

changes — no Refresh button is needed. Dropdown options are rebuilt automatically whenever new departments or managers appear in the source data.

Note: The DrillDown tab preserves the current filter selections between refreshes. Only the results table updates — the filter values HR has chosen remain in place.

5. How HR Uses the Web App Daily

The web app is a browser-based executive dashboard served at a fixed Google URL. HR signs in once with their Google account and sees live, automatically updating data.

Web App URL

https://script.google.com/macros/s/AKfycbwy09xbXpKmRIstCaqpUczxI9Q5WACvQ_8ADU-Jsy0BtP23hoaMYiHBWv66g27yLPde/exec

Sections in the Web App

Section	Description
KPI cards	Total employees, India, US, confirmed, probation, interns, high risk, productivity flags, LWD alerts, probation alerts. Color-coded for quick scanning (green = healthy, orange/yellow = needs attention). LWD alerts (orange) and probation alerts (yellow) shown as individual cards with employee name, ID, department, region, relevant date, and days remaining.
Active Alerts	Bar chart showing headcount per department.
Department Breakdown	Exit count and attrition rate per quarter, calculated live from the Offboarded Resources tab.
Quarterly Attrition	Table with color-coded risk level and status badges.
Risk Register	Visual top-down tree — root boxes for top managers, lighter boxes for mid-level managers, and gray boxes for individual contributors. Horizontally scrollable.
Org Chart	Region, Department, Manager, and Status dropdown filters. Results update instantly. Shows all matching employees with ID, name, department, designation, manager, and status.
Employee Drill-Down	

How the Web App Stays Up to Date

The web app refreshes itself automatically — HR does not need to click anything for routine updates. See Section 7.5 for the full explanation of the auto-refresh mechanism. A manual Refresh control is also available in the top-right corner for HR to force an immediate reload if desired (for example, immediately after making a bulk edit).

6. Adding Employees, Departments & Projects

Adding a New Employee

1. Open the India Employee Database or US Employee Database tab, whichever is appropriate.
2. Scroll to the first empty row below the existing data.

3. Fill in all required fields: Employee ID, Employee Name, Department, Designation, Reporting Manager, Skillset, Date of Joining, and Employment Status.
4. For interns, also fill in the Last Working Day (Interns Only) field.
5. For US employees, also fill in Current Allocation (%) and CTC (Annual USD).
6. Press Enter or click another cell. The onEdit trigger fires automatically.
7. Within seconds, the Dashboard tab, OrgChart tab, and DrillDown tab all update, and the web app reflects the change on its next automatic refresh (within about 1.5 seconds).
8. If the new employee is an intern with a Last Working Day within 45 days, they appear in LWD alerts immediately.
9. If the new employee is Under Probation with a confirmation date within 30 days, they appear in probation alerts immediately.

Adding a New Department

- Type the new department name directly into the Department column for the relevant employee row.
- The dashboard's Department Breakdown updates automatically.
- The DrillDown Department dropdown picks up the new department automatically on the next refresh.
- No code changes are required.

Adding a New Manager (Org Chart)

- Add the manager's name in the Reporting Manager column for their direct reports.
- The OrgChart tab rebuilds automatically and the new manager appears in the hierarchy.
- If the manager is not listed as a direct report under anyone else, they become a root node at the top level.

Employment Status Values

The Employment Status column must be one of exactly three values (case-sensitive):

Value	Meaning
Confirmed	Employee has passed probation and is confirmed.
Under Probation	Employee is within the 180-day probation period. Confirmation date = Date of Joining + 180.
Intern	Intern with a defined Last Working Day. LWD alerts fire within 45 days of that date.

Note: Entering any value other than these three (for example 'Active' or 'Full Time') triggers a validation message, and the employee will not appear correctly in status-based filters or alerts until corrected.

7. Real-Time Reflection Pattern

The system uses an installable onEdit trigger as the core mechanism that makes the Sheet dashboard update automatically whenever HR edits source data, combined with client-side polling that keeps the web app in sync. Together these mean HR only ever needs to edit data — the system handles everything else.

How It Works — Step by Step

10. HR edits a cell. HR types into any source tab (India/US Employee Database, Risk Report, Productivity, Offboarded Resources, and so on) and presses Enter.
11. onEditInstallable fires. Apps Script detects the edit through the installable trigger, which is set up once by running createInstallableTriggers().
12. Validation runs. validateEmployeeEdit() checks the edited cell(s) for required fields, valid dates, and correct Employment Status values, and surfaces a clear message to HR if something is wrong (see Section 10).
13. refreshAll() runs. clearCache() empties the in-memory cache, then renderDashboard(), renderOrgChart(), and setupDrillDownFilters() / renderDrillDown() are called in sequence, each wrapped in its own try/catch so one failure does not stop the others.
14. Both surfaces update. The Sheet Dashboard tab is rewritten with fresh data. The DrillDown tab filters and results are rebuilt. The web app picks up the change on its next automatic poll.
15. Logging. Every run is logged to the Logs tab with a timestamp, level (INFO, WARN, or ERROR), and a descriptive message.

DrillDown Filter Changes

When HR changes a dropdown filter in the DrillDown tab (cells B2, D2, F2, and H2), the trigger detects this specific edit and calls renderDrillDown() directly, skipping the full pipeline so the response stays fast.

Daily Trigger

A separate time-based trigger fires dailyPipelineRun() once per day at approximately 8 AM. This runs the full pipeline, sends the email digest if any alerts are active, and writes a Changelog entry. This guarantees both surfaces are fully up to date each morning even if no edits were made the previous day.

7.5 Web Application Auto-Refresh (Client-Side Polling)

The web application implements automatic client-side refresh through polling every 1500 milliseconds (1.5 seconds). A short script running in the browser repeatedly calls getWebAppData() in the background and redraws the affected sections whenever the returned data has changed. This ensures HR users see near real-time updates without manually refreshing or reloading the page.

Whenever HR performs an action such as adding an employee, updating an employment status, modifying a risk record, updating a reporting relationship, renaming or reordering columns, or adding an entry to Offboarded Resources, the backend pipeline processes that change through the onEdit trigger and the refresh functions described above. Depending on the size of the edit and Google Apps Script's execution time for that run, the change may take a few seconds to propagate fully across all dashboard components — typically well under the 1.5-second polling interval for single-row edits, and slightly longer for bulk edits or full pipeline runs.

All update activity is recorded in the Logs tab, giving complete visibility into processing status and execution history. While minor processing delays may occur during larger updates or structural changes (such as reordering many columns at once), the system consistently converges on accurate, up-to-date information across both the Google Sheets dashboard and the deployed web application.

This design balances performance, reliability, and real-time visibility: HR always works with the latest available data, while the system itself avoids unnecessary load by only recomputing what is needed and only as often as is useful.

Note: Because the web app polls every 1.5 seconds, the manual Refresh button described in Section 5 is a convenience for forcing an immediate reload — it is not required for normal use.

8. Header-Agnostic Column Lookup

The pipeline never references columns by position, by spreadsheet column letter, or by a hardcoded index. All column access goes through a three-tier resolution system inside DataLayer.gs. This means HR can rename column headers — even to unexpected names — without breaking the pipeline.

Three-Tier Resolution Order

16. Exact-match map. A lookup table of known header variants, for example 'Emp ID' to 'Employee ID', 'DOJ' to 'Date of Joining', 'Current Allocation (%)' to 'Allocation %', and 'CTC (Annual USD)' to 'CTC'. Over forty common variants are covered.
17. Fuzzy keyword match. If no exact match is found, the header is checked for keywords. For example, any header containing both 'emp' and 'id' resolves to 'Employee ID', and any header containing 'alloc' resolves to 'Allocation %'. This handles creative renames that are not in the exact map.
18. Positional fallback. If a header matches neither tier (for example, a column renamed to something arbitrary like 'XYZ'), the system falls back to that column's expected position for the tab, defined in COLUMN_POSITION_FALLBACK. Since reordering columns in practice usually means renaming the header while the data stays in its original column, this positional map preserves correct field mapping even for unrecognized names.
19. Unknown columns are kept, not dropped. If a header does not match any tier and there is no positional fallback entry for that index, the raw header name is kept as-is and the column's data is preserved rather than silently discarded.

Positional Fallback Map

The COLUMN_POSITION_FALLBACK constant in DataLayer.gs defines the expected column order for each tab. This must match the actual sheet column order. If columns are physically reordered (not just renamed), this map must be updated — see Section 19.

Tab	Expected Column Order
India Employee Database	Employee ID, Employee Name, Department, Designation, Reporting Manager, Skillset, Date of Joining, Employment Status, Last Working Day (Interns Only)
US Employee Database	Employee ID, Employee Name, Department, Designation, Reporting Manager, Allocation %, CTC, Skillset, Date of Joining, Employment Status, Last Working Day (Interns Only)
Risk Report	Employee Name, Risk Category, Risk Level, Status, Mitigation Action
Finance	Employee ID, Employee Name, Department, CTC
Productivity	Employee ID, Employee Name, Department, Overall Avg Hrs/Day, Below 8 Hrs Flag
Offboarded Resources	Employee ID, Employee Name, Department, Designation, Reporting Manager, Date of Joining, Last Working Day (Interns Only), Employment

Tab	Expected Column Order
	Status, Region
RM Data	Employee ID, Employee Name, Department (plus monthly Project / Allocation % column pairs)

Note: The US Employee Database column order differs from India — Allocation % and CTC appear at columns 6 and 7, before Skillset. This is correctly mapped in the positional fallback. Do not reorder these columns without updating COLUMN_POSITION_FALLBACK to match.

9. In-Memory Cache Layer

The system uses a request-scoped in-memory cache to avoid reading the same sheet data multiple times during a single pipeline run. This significantly reduces Google Sheets API calls and execution time.

Component	Role
_cache object	A plain JavaScript object (<code>_cache = {}</code>) declared at the top of <code>DataLayer.gs</code> . Stores loaded data keyed by tab name or dataset name for the duration of one execution.
clearCache()	Called at the start of every <code>refreshAll()</code> run. Empties the cache so each pipeline run reads fresh data from the sheet.
getCached(key, fn)	If <code>_cache[key]</code> already exists, returns it immediately. Otherwise calls the loader function <code>fn()</code> , stores the result in the cache, and returns it.
cachedEmployees()	Wraps <code>getAllEmployees()</code> . The first caller in a pipeline run reads the India and US tabs (two API calls). All subsequent callers in the same run reuse the cached result.
cachedSheetData(tab)	Same pattern applied to Risk Report, Productivity, Finance, and Offboarded Resources tabs.

Performance Impact

Scenario	API Calls per Refresh
Before caching	<code>getAllEmployees()</code> called four or more times per refresh — eight or more API calls to Sheets.
After caching	<code>getAllEmployees()</code> called once — two API calls. All other functions read from memory for the rest of the run.

Note: The cache is an Apps Script execution-scope variable. It never persists between separate trigger executions, so every `onEdit` gets a completely fresh data set immediately after `clearCache()` runs at the start of `refreshAll()`.

10. Input Validation

When HR enters invalid data, the system shows a clear, human-readable message in Google Sheets. No stack trace is ever shown to HR. Validation runs on every cell edit and on paste operations across all source tabs.

Validated Tabs and Required Fields

Tab	Required Fields
India Employee Database	Employee ID, Employee Name, Department, Employment Status, Date of Joining
US Employee Database	Employee ID, Employee Name, Department, Employment Status, Date of Joining
Risk Report	Employee Name, Risk Category, Risk Level
Offboarded Resources	Employee ID, Employee Name, Department, Date of Joining
Finance	Employee ID, Employee Name, CTC
Productivity	Employee ID, Employee Name, Overall Avg Hrs/Day

Validation Rules

Field Type	Rule and Message
Required text fields	Cannot be blank. Message: "Row X: '[Field Name]' is required and cannot be blank."
Date fields	Date of Joining and Last Working Day must parse as valid dates. Message: "Row X: '[Field Name]' must be a valid date (for example 15/0 1/2 024)."
Employment Status	Must be exactly Confirmed, Under Probation, or Intern. Message: "Row X: 'Employment Status' must be Confirmed, Under Probation, or Intern."
Risk Level	Must be High, Medium, Low, or Critical.
Allocation %	Must be a number between 0 and 100.
CTC	Must be a positive number.
Overall Avg Hrs/Day	Must be a number between 0 and 24.

Note: Validation also handles paste operations. If HR pastes ten rows at once, every row is checked and all issues are reported together rather than one at a time.

11. Alerts System

LWD Alerts (Intern Last Working Day)

Surfaces interns whose Last Working Day falls within LWD_ALERT_DAYS (default 45) days of today — covering both upcoming and just-passed cases.

Aspect	Detail
Log ic	For each employee with Employment Status = 'Intern', read Last Working Day. Calculate daysUntil = LWD minus today. Include the employee if daysUntil is between -45 and +45 inclusive. Upcoming (daysUntil > 0), Today (daysUntil = 0), Just Passed (daysUntil < 0).
Status labels	LWD_ALERT_DAYS in the _Config tab. Default 45.
Threshold	Both the Sheet Dashboard (Recent Alerts section) and the Web App (Active Alerts section).
Displayed	

Probation Alerts

Surfaces employees whose confirmation date (Date of Joining plus 180 days) falls within PROBATION_ALERT_DAYS (default 30) days of today.

Aspect	Detail
Log ic	For each employee with Employment Status = 'Under Probation', read Date of Joining. Calculate confirmationDate = DOJ + PROBATION_PERIOD_DAYS, then daysUntil = confirmationDate minus today. Upcoming Confirmation, Confirmation Due Today, Confirmation Overdue.
Status labels	PROBATION_ALERT_DAYS and PROBATION_PERIOD_DAYS in the _Config tab.
Threshold	
Key rule	No dates are ever hardcoded. All date math uses new Date(), equivalent to TODAY(). Changing an employee's Date of Joining automatically recalculates their confirmation date and alert status on the next refresh.

12. Org Chart

The OrgChart tab is a hierarchical representation of the reporting structure, rebuilt from the Reporting Manager column on every pipeline run. No manual drawing or updating is ever needed.

Data source

	Detail
	The Reporting Manager column in both the India and US Employee Database tabs. Managers who do not themselves appear as a direct report under any other manager are treated as root nodes and shown at the top level of the tree.
Root detection	A text-based indented tree using Unicode box-drawing characters.
Rendering — Sheet	
Rendering — Web App	A visual top-down tree. Root managers are shown in darker boxes, mid-level managers in lighter boxes, and individual contributors in gray boxes.
Auto-rebuild	Fires on every onEdit to a source tab. If HR changes a Reporting Manager value, the chart redraws automatically on the next refresh.
Web App chart interaction	Horizontally scrollable. Each manager box shows the manager's name and number of direct reports, with a hover effect for emphasis.

13. Drill-Down Filters

The DrillDown tab in the Sheet and the Employee Drill-Down section in the Web App both allow HR to filter the full employee list by Region, Department, Reporting Manager, and Employment Status.

	Behaviour
Sheet DrillDown	Dropdowns in row 2 (B2 = Region, D2 = Department, F2 = Reporting Manager, H2 = Status). Changing any dropdown value triggers renderDrillDown(), which rebuilds the results table below.
Web App Drill-Down	Four select dropdowns rendered in the browser. Changing a dropdown fires updateFilter(), which filters the already-loaded employee data client-side for an instant response, and is also picked up by the next automatic poll.

Surface	Behaviour
Filter values	Dropdown options are built dynamically from the actual data via <code>getUniqueValues()</code> . New departments, managers, and statuses are added to the dropdown options automatically — nothing is hardcoded. The Sheet DrillDown preserves the current filter values on every refresh.
Preserving selections	<code>setupDrillDownFilters()</code> checks the existing dropdown values before rebuilding the option lists, so HR's chosen filters are not reset.
All option	Selecting 'All' for any filter (All Regions, All Departments, All Managers, All Statuses) removes that filter and shows the full employee list, subject to any other filters still applied.

14. Email Digest

The system sends an HTML email digest to the configured recipient summarising all active LWD and probation alerts. No email is sent if there are no active alerts.

Aspect	Detail
Recipient	<code>ALERT_EMAIL_RECIPIENT</code> in the <code>_Config</code> tab. Default: <code>hr@techolution.com</code> . Multiple recipients can be separated by commas.
Triggered by	<code>dailyPipelineRun()</code> , which runs once daily at approximately 8 AM. The email is not sent on every <code>onEdit</code> — only on the daily trigger.
Content	An HTML email with two sections: an Intern LWD Alerts table and a Probation Confirmation Alerts table. Each row shows employee ID, name, department, region, the relevant date, and the alert status.
Skip condition	If both <code>lwdAlerts.length</code> and <code>probationAlerts.length</code> are zero, the email is skipped for that day — HR simply does not receive a digest when there is nothing to report.
Failure handling	If <code>MailApp.sendEmail()</code> throws an error (for example, daily quota exhaustion), the error is caught and logged to the Logs tab as an ERROR — the pipeline itself continues normally.

Note: HR receives this alert digest automatically by email every day at approximately 8 AM, with no action required. The email summarises all interns approaching their Last Working Day and all employees approaching their confirmation date, so HR always has a same-day view of what needs attention without opening the Sheet or the web app.

15. Automation & Triggers

Two triggers are active in this project. Both are set up at once by running `createInstallableTriggers()` manually from the Apps Script editor. Re-running this function is always safe — it deletes any existing triggers from the project first, preventing duplicates.

Trigger	Type	Function	Purpose
Real-time reflection	Installable <code>onEdit</code> , bound to the spreadsheet	<code>onEditInstallable(e)</code>	Fires on every edit to the spreadsheet. Routes to <code>renderDrillDown()</code> if the edit was a DrillDown filter change, or to the full <code>refreshAll()</code> pipeline for edits to any source tab. Runs with full permissions, so it can send email

Trigger	Type	Function	Purpose
			and access external services if needed.
Daily pipeline	Time-based, daily at approximately 8 AM	dailyPipelineRun()	Runs refreshAll(), then sendAlertDigest(), then writeChangelog(). Guarantees both surfaces are fresh every morning even with no prior edits.

Note: If triggers are ever deleted or stop firing, run createInstallableTriggers() once from the Apps Script editor. Open Apps Script → Triggers to verify that both triggers are listed and active.

16. Logs & Changelog

Logs Tab

An append-only log of every pipeline action, with three columns: Timestamp, Level (INFO, WARN, or ERROR), and Message. This tab is never cleared automatically and grows continuously as a full audit trail.

Level	Meaning
INFO	Normal operations — dashboard rendered, filters initialized, alert counts calculated, trigger source recorded.
WARN	Non-fatal issues — an unrecognized column header was resolved via positional fallback, a validation issue was found in an HR edit, or a tab expected by a function was missing.
ERROR	Failures such as renderDashboard failing, sendAlertDigest failing, or onEditInstallable throwing an exception. The pipeline continues running other sections even when one section logs an error.

Changelog Tab

Appended by dailyPipelineRun() on every daily trigger run. Columns: Run ID, Timestamp, Source, Duration (ms), LWD Alert Count, Probation Alert Count, and Email Status. This enables trend analysis of alert counts and pipeline performance over time.

Note: HR does not need to check the Logs or Changelog tabs day to day. The daily 8 AM alert email (see Section 14) already delivers the information HR needs directly to their inbox. Logs and Changelog exist for the admin/developer to audit pipeline health.

17. Modifying Thresholds & Config

All thresholds live in the _Config tab. No code changes are needed to adjust alert windows, the probation period, the productivity target, or email recipients.

How to Change a Threshold

20. Open the Google Sheet.
21. Navigate to the _Config tab.

22. Find the Key in column A that needs to change.
23. Edit the corresponding Value in column B.
24. The next pipeline run — either triggered by an onEdit or by the daily trigger — will use the new value automatically.
25. No script deployment or code change is required.

Examples

Change the LWD alert window from 45 to 60 days

Change to Make	
Set LWD_ALERT_DAYS = 60 in _Config.	
Set ALERT_EMAIL_RECIPIENT = hr@techolution.com, manager@techolution.com (comma-separated).	
Set PROBATION_PERIOD_DAYS = 90. All confirmation dates and probation alerts recalculate automatically on the next refresh.	
Set PRODUCTIVITY_TARGET_HRS = 7.	

18. Troubleshooting Guide

Symptom	Resolution
Editing a source tab does not update the Dashboard	Go to Apps Script → Triggers and verify that onEditInstallable exists for this project. If it is missing, run createInstallableTriggers() once from the editor. Run setupDrillDownFilters() manually from the editor. Also confirm that Employment Status values in the sheet exactly match Confirmed, Under Probation, or Intern, since unexpected values can prevent options from populating correctly.
DrillDown dropdowns show outdated or missing options	Verify Employment Status is exactly 'Intern' (for LWD) or 'Under Probation' (for probation). Verify the relevant date is stored as a real date, not text. Check that LWD_ALERT_DAYS or PROBATION_ALERT_DAYS in _Config is large enough to cover the date in question.
An expected LWD or probation alert is not showing	Verify that createHtmlOutputFromFile('WebApp1') in WebApp.gs matches the HTML file name exactly. Redeploy via Deploy → Manage deployments → Edit → New version → Deploy.
Web app shows an old version or is not redeployed	Open Apps Script → Executions to see the error and stack trace. Check the Logs tab for ERROR entries around the same timestamp. Common causes include a missing tab name, getConfig() failing because the _Config tab is missing, or a tab's headers being unexpectedly empty.
A pipeline run throws an exception	If there were no active LWD or probation alerts that day, no email is sent by design. If alerts were active but the email still did not arrive, check the Logs tab for an ERROR entry around 8 AM mentioning sendAlertDigest — MailApp has a daily sending quota of 100 emails for consumer Google accounts, and higher for Workspace accounts.
The daily alert email did not arrive	Check the Logs tab for WARN entries mentioning 'positional fallback'. This means the renamed header was not recognized by the exact-match or fuzzy-match tiers. Either add the new header text as

Symptom	Resolution
	an alias in <code>normalizeHeader()</code> , or confirm the positional fallback for that tab still matches the actual column order (see Section 19). Make any edit to a source tab to trigger <code>refreshAll()</code> , which calls <code>setupDrillDownFilters()</code> and rebuilds the dropdown option lists. Alternatively, run <code>setupDrillDownFilters()</code> manually.
DrillDown dropdowns do not include a newly added department or manager	Check the Logs tab for a 'renderOrgChart failed' error. Verify that all Reporting Manager values are non-empty text strings — empty manager fields can prevent an employee from being placed correctly in the tree.
Org chart is missing an employee or shows an incorrect hierarchy	Ensure Date of Joining and Last Working Day are stored as actual date values, not text. Apps Script reads Google Sheets date objects directly; text that merely looks like a date may not be parsed correctly and will be skipped by validation.

19. Change Management & Version Control

Making Code Changes

26. Open the Google Sheet → Extensions → Apps Script.
27. Make changes to the relevant .gs file.
28. Save with Ctrl+S.
29. For web app changes, also update WebApp1.html.
30. After any change, redeploy the web app: Deploy → Manage deployments → Edit → New version → Deploy.
31. Test the change by running `testTriggersPipeline()` manually from the editor.
32. Check the Logs tab to confirm no new errors were produced.

Re-running Triggers Setup

If triggers are ever deleted or stop working, run `createInstallableTriggers()` once manually. This safely removes all existing triggers for the project first and then creates fresh ones.

Adding New Column Aliases

If HR renames a column to something not covered by the existing exact-match map or fuzzy matching:

33. Open `DataLayer.gs`.
34. Find the `normalizeHeader()` function.
35. Add the new alias to `exactMap`, for example: "My Custom ID": "Employee ID".
36. Save and test by running `testDataLayer()`.
37. No redeployment is needed for Sheet dashboard changes. Redeploy the web app only if `getWebAppData()` is affected.

Updating the Positional Fallback

If columns are physically reordered in a sheet (not just renamed):

38. Open `DataLayer.gs`.

39. Find the COLUMN_POSITION_FALLBACK constant.
40. Update the array for the affected tab to match the new column order.
41. The order must exactly match the actual sheet, left to right.

20. Apps Script Code Maintenance

File Structure

	Responsibilities
Data Layer.gs	Header-agnostic data access layer: normalizeHeader(), getSheetData(), getField(), joinOn(), getAllEmployees(), cachedEmployees(), cachedSheetData(), the in-memory cache, and COLUMN_POSITION_FALLBACK.
Alerts.gs	getLWDAAlerts(), getProbationAlerts(), sendAlertDigest(). All alert logic and the daily email digest.
Dashboard.gs	renderDashboard(). Writes the Sheet Dashboard tab from source data.
DrillDown.gs	setupDrillDownFilters(), renderDrillDown(), getUniqueValues(), setDropdown(). Sheet DrillDown tab.
OrgChart.gs	renderOrgChart(), writeNode(), plus helper functions getRiskRecords() and getProductivityFlags(). Sheet OrgChart tab.
Triggers.gs	onEditInstallable(), refreshAll(), validateEmployeeEdit(), dailyPipelineRun(), writeChangelog(), createInstallableTriggers(). All trigger and pipeline orchestration.
Config.gs	getConfig() reads the _Config tab. Also contains the Logs.gs functions logInfo(), logWarn(), logError(), and logEvent().
WebApp.gs	doGet(), getWebAppData(), getQuarterlyAttrition(), buildOrgData(), formatDate(). Web app backend.
WebApp1.html	Web app frontend: HTML, CSS, and JavaScript, including the 1.5-second polling loop. Renders all dashboard sections from the JSON returned by getWebAppData().

Key Design Principles

- No hardcoded dates — all date math uses new Date(), the equivalent of TODAY().
- No hardcoded column indices — columns are always resolved via normalizeHeader() and COLUMN_POSITION_FALLBACK.
- No hardcoded thresholds — all thresholds are read from _Config via getConfig().
- Cache-first reads — pipeline functions use cachedEmployees() and cachedSheetData(), never getAllEmployees() directly.
- Error isolation — every section inside refreshAll() is wrapped in its own try/catch so a single failure does not stop the rest of the pipeline.
- Append-only logging — every run, action, and error is logged. Logs are never cleared by the pipeline.
- Human-readable errors — validation issues and pipeline failures surface as clear messages to HR, never as raw stack traces.

Test Functions

Function	Purpose
testDataLayer()	Verifies India and US employee loading, field resolution, join operations, and column index lookups.
testAlerts()	Prints current LWD and probation alerts to the Apps Script console.
testTriggersPipeline()	Runs a full refreshAll() cycle and prints the duration and alert counts.
testConfig()	Prints all _Config values to verify they loaded correctly.
testLogs()	Writes test INFO, WARN, and ERROR entries to the Logs tab.
testSendAlertDigest()	Sends a real email digest to the configured recipient. Use with care — this sends an actual email.
	One-time setup: removes old triggers and creates fresh onEdit and daily triggers.
createInStallableTriggers()	

Document Information

Field	Value
Developer	Lokineni Hindhuja
Organisation	Techolution
Stack	Google Sheets + Apps Script (V8 runtime) + HtmlService
Date	14 June 2026
Web App URL	https://script.google.com/macros/s/AKfycbwY09xbXpKmRIsTcaqPUczxI9Q5WACvQ_8ADU-Jsy0BtP23hoaMYiHBWv66g27yLPde/exec
Alert Email	hr@techolution.com

Summary

This SOP covers the full HR Automation Dashboard system: the two-surface architecture (Sheet dashboard and web app), how HR uses both surfaces day to day, how new employees, departments, and managers are added, the real-time reflection pattern including the web app's 1.5-second auto-refresh, header-agnostic column handling for renamed or reordered columns, the in-memory cache layer, input validation, the alerts and daily email digest system, the org chart and drill-down filters, automation triggers, logging, configuration, troubleshooting, and code maintenance guidance for future developers.

For day-to-day use, HR needs only Sections 4 through 6 and the daily 8 AM email digest described in Section 14. Sections 7 through 20 are intended for the developer or admin maintaining the system.